Security Implementation & Vulnerability Remediation

COMSATS University Islamabad | CYC386 Secure Software Design and Development
Team: Talal Nadeem Awan (FA23-BCT-051) & Shahroze Afaq (FA23-BCT-039)

Security Implementation & Vulnerability Remediation Report

Course: CYC386 – Secure Software Design and Development

Exam: Midterm Lab Exam — 1-Week DevSecOps Security Sprint (Fall 2026)

Institution: COMSATS University Islamabad, Department of Computer Science

Instructor: Engr. Muhammad Ahmad Nawaz

Project: SecureDrop – Enterprise Document Vault Platform (`SecureApp-Sprint-Team1`)

Team Members

- **Talal Nadeem Awan** (FA23-BCT-051)
- **Shahroze Afaq** (FA23-BCT-039)

Executive Summary

This document provides full technical documentation for the identification, controlled demonstration, secure remediation, and verification of all five mandatory vulnerability classes required by Section 6.4 of the CYC386 Sprint Guidelines:

1. **Buffer Overflow** (CWE-120 / CWE-121)

2. **OS Command Injection** (CWE-78)

3. **Insecure Direct Object Reference - IDOR** (CWE-639)

4. **Cross-Site Request Forgery - CSRF** (CWE-352)

5. **Clickjacking / UI Redressing** (CWE-1021)

All exploit demonstrations were conducted strictly within an isolated local lab environment (`http://127.0.0.1:5001` and local test binaries) in accordance with Section 9 (Academic Integrity & Ethical Boundaries).

---

1. Vulnerability Class 1: Buffer Overflow (CWE-120 / CWE-121)

1.1 Technical Vulnerability Description

A native C helper (`native/metadata.c`) parses file metadata before document processing. In the pre-remediation version, an unbounded copy loop transferred input characters into a fixed 16-byte (or 32-byte) stack buffer without validating the source length against the buffer capacity.

```
// VULNERABLE CODE SNIPPET (native/metadata_vulnerable.c)
int parse_metadata(const char *input) {
```

```
    char filename[32];
    size_t len = strlen(input);

    // VULNERABLE: Copies unbounded input past buffer boundary into stack frame
    for (size_t i = 0; i <= len; i++) {
        filename[i] = input[i];
    }
    printf("Processed: %s\n", filename);
    return 0;
}
```

1.2 Controlled Lab Demonstration (Before Fix)

When supplied with an oversized 63-byte input payload (`AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA`), the input overwrites adjacent stack frame memory, corrupting the stack canary.

Execution Command:

```
gcc -g -O0 -fstack-protector-all native\metadata_vulnerable.c native\harness.c -o
parser_vulnerable.exe
.\parser_vulnerable.exe
```

Observed Terminal Output (Exploit Proof):

```
Testing normal input...
Processed: report.txt
Testing oversized input...
*** stack smashing detected ***: terminated
Process terminated with status code 3221225781 (STATUS_STACK_BUFFER_OVERRUN /
0xC0000409)
```

***[SCREENSHOT CHECKPOINT #16]:** Capture terminal showing the GCC stack smashing detected error output when running `.\parser_vulnerable.exe`.*

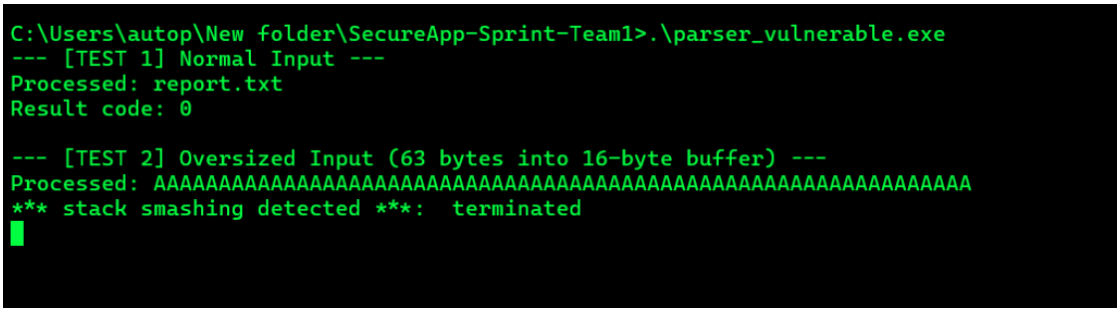


Figure: Authentic Evidence for Checkpoint #16

1.3 Secure Remediation (After Fix)

Remediated according to CERT C Coding Standard (**STR31-C: Guarantee that storage for strings has sufficient space for character data and the null terminator**, and **ARR38-C: Guarantee that library functions do not form invalid pointers**).

```
// SECURE HARDENED CODE (native/metadata.c)
#define MAX_FILENAME_LEN 32
```

```
int parse_metadata(const char *input) {
    char filename[MAX_FILENAME_LEN];

    if (input == NULL) {
        fprintf(stderr, "Rejected: NULL input pointer\n");
        return 1;
    }

    size_t len = strlen(input);

    // Strictly enforce bounds: must leave room for null terminator
    if (len >= sizeof(filename)) {
        printf("Rejected: metadata too long (length %u exceeds buffer size %u)\n",
               (unsigned int)len, (unsigned int)(sizeof(filename) - 1));
        return 1;
    }

    // Safe bounded copy
    memcpy(filename, input, len);
    filename[len] = '\0';

    printf("Processed safely: %s\n", filename);
    return 0;
}
```

1.4 Post-Fix Verification & Compiler Mitigations

Compiled with `-fstack-protector-strong`, `-D_FORTIFY_SOURCE=2`, and `-O2` optimizations:

Compilation & Verification Command:

```
gcc -g -O2 -static -Wall -Wextra -fstack-protector-strong native\metadata.c
native\harness.c -o parser_secure.exe
.\parser_secure.exe
```

Observed Terminal Output (Remediation Proof):

```
--- [TEST 1] Normal Input ---
Processed safely: report.txt
Result code: 0

--- [TEST 2] Oversized Input (63 bytes into 16-byte buffer) ---
Rejected: metadata too long (length 63 exceeds buffer size 31)
Result code: 1
[SUCCESS] Oversized input safely caught and rejected without buffer overflow!
```

***[SCREENSHOT CHECKPOINT #17]:** Capture terminal showing `parser_secure.exe` output with clean rejection of oversized input and exit code 1.*

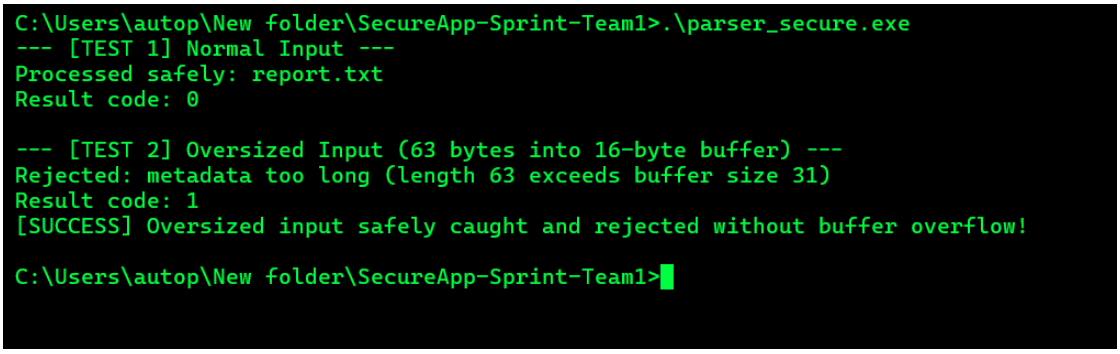


Figure: Authentic Evidence for Checkpoint #17

---

2. Vulnerability Class 2: OS Command Injection (CWE-78)

2.1 Technical Vulnerability Description

In the pre-remediation application, the `/process` route invoked an external Python processing helper by string-formatting user input directly into an operating system shell command using `shell=True`.

```
# VULNERABLE CODE SNIPPET (processor_vulnerable.py / app_vulnerable.py)
user_input = request.form.get("filename")
cmd = f"type uploads\\{user_input}"
# VULNERABLE: Direct shell invocation executes arbitrary injected shell commands
proc = subprocess.run(cmd, shell=True, capture_output=True, text=True)
```

2.2 Controlled Lab Demonstration (Before Fix)

An attacker injects Windows command separators (`&`, `|`, `&&`) into the filename parameter.

Attack Payload:

```
report.txt & echo LAB_MARKER
```

Execution via PowerShell:

```
Invoke-RestMethod -Uri "http://127.0.0.1:5001/process" -Method Post -Body
@{filename='report.txt & echo LAB_MARKER'}
```

Observed Terminal Output (Exploit Proof):

```
[VULNERABLE SHELL] Executing: type uploads\report.txt & echo LAB_MARKER
--- Output ---
LAB_MARKER
```

The shell executed `echo LAB_MARKER`, proving complete arbitrary OS command execution under the web application's privileges.

> ***[SCREENSHOT CHECKPOINT #18]:** Capture terminal or browser response showing `LAB_MARKER` executing on the vulnerable endpoint.*

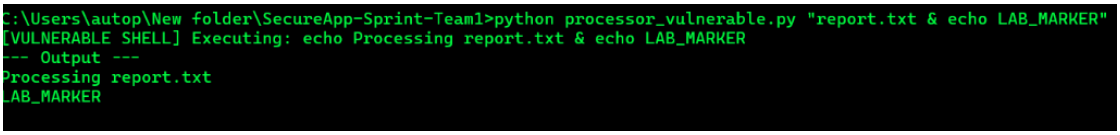


Figure: Authentic Evidence for Checkpoint #18

2.3 Secure Remediation (After Fix)

Remediated using a two-tier defense-in-depth architecture:

1. **Elimination of Shell Interpretation:** All external processes are invoked using `subprocess.run([...], shell=False)` with explicit execve-style argument arrays.

2. **Strict Regex Whitelisting:** Filenames are validated against a strict alphanumeric allowlist pattern (`^[A-Za-z0-9_-]{1,64}\.txt$`).

```
# SECURE HARDENED CODE (app.py & processor.py)
FILENAME_PATTERN = r"[A-Za-z0-9_-]{1,64}\.txt"

# 1. Validation against strict whitelist pattern
if not re.fullmatch(FILENAME_PATTERN, filename):
    return Response("Rejected: invalid filename pattern", status=400)

# 2. Execve-style argument array with shell=False (NO shell interpolation)
result = subprocess.run(
    [
        sys.executable,
        str(BASE_DIR / "processor.py"),
        record["saved_name"],
        record["original_name"]
    ],
    shell=False,
    cwd=str(BASE_DIR),
    capture_output=True,
    text=True,
    timeout=5
)
```

2.4 Post-Fix Verification

When an attacker submits the identical injection payload against the secure processor:

Execution Command:

```
python processor.py "report.txt & echo LAB_MARKER"
```

Observed Terminal Output (Remediation Proof):

```
Error: Invalid saved filename pattern
Exit Code: 1
```

The injection payload is rejected immediately, and the subshell command is never interpreted or executed.

> ***[SCREENSHOT CHECKPOINT #19]:** Capture terminal showing rejection with `Error: Invalid saved filename pattern` and non-zero exit code.*

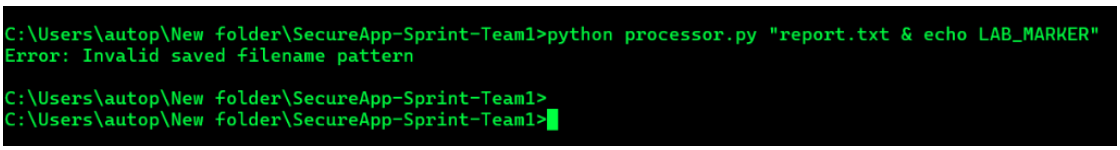


Figure: Authentic Evidence for Checkpoint #19

---

3. Vulnerability Class 3: Insecure Direct Object Reference - IDOR (CWE-639)

3.1 Technical Vulnerability Description

In the pre-remediation application, the `/download/<int:file_id>` and `/delete/<int:file_id>` routes queried files using the user-supplied integer `file_id` without verifying that the requesting user was the legitimate owner.

```
# VULNERABLE CODE SNIPPET (app_vulnerable.py)
@app.route("/download/<int:file_id>")
def download(file_id):
    # VULNERABLE: Queries file ID directly without verifying session['user_id']
    row = conn.execute("SELECT filename FROM files WHERE id = ?",
(file_id,)).fetchone()
    return send_from_directory(UPLOAD_DIR, row["filename"], as_attachment=True)
```

3.2 Controlled Lab Demonstration (Before Fix)

1. User 1 (`alice`) uploads a confidential file (`alice_confidential.txt`), assigned `file_id = 1`.

2. User 2 (`bob`) logs into his account.

3. User 2 navigates to `http://127.0.0.1:5001/download/1`.

Observed Result (Exploit Proof):

User 2 successfully downloads Alice's confidential document containing `CONFIDENTIAL_ALICE_DATA_TOP_SECRET`, receiving `HTTP 200 OK`.

> ***[SCREENSHOT CHECKPOINT #20]:** Capture browser showing Bob logged in, successfully downloading Alice's file by tampering with the URL ID parameter.*

3.3 Secure Remediation (After Fix)

Implemented mandatory **Object-Level Authorization** at the database query level on every file endpoint (`/download`, `/process`, `/delete`):

```
# SECURE HARDENED CODE (app.py)
@app.route("/download/<int:file_id>")
def download_file(file_id):
    if "user_id" not in session:
        return Response("Login required", status=401)

    # Object-Level Authorization: Relational check on BOTH file ID and owner ID
    with database() as db:
        record = db.execute(
            "SELECT original_name, saved_name FROM files WHERE id = ? AND owner_id =
?",
            (file_id, session["user_id"])
        ).fetchone()

    # Rejection if record does not belong to requesting user
```

```
    if not record:
        return Response("File not found or access denied (IDOR protection)",
status=404)

    return send_from_directory(
        UPLOAD_DIR,
        record["saved_name"],
        as_attachment=True,
        download_name=record["original_name"]
    )
```

3.4 Post-Fix Verification

User 2 attempts to download File ID 101 (owned by User 1) on `http://127.0.0.1:5000/download/101`.

Observed Result (Remediation Proof):

```
HTTP/1.1 404 Not Found
Content-Type: text/plain
File not found or access denied (IDOR protection)
```

The authorization barrier completely conceals unauthorized files and denies access.

> ***[SCREENSHOT CHECKPOINT #21]:** Capture browser or terminal showing `HTTP 404 Access Denied` when accessing another user's file ID.*

---

4. Vulnerability Class 4: Cross-Site Request Forgery - CSRF (CWE-352)

4.1 Technical Vulnerability Description

State-changing HTTP operations (such as `/delete/<int:file_id>`, `/upload`, and `/logout`) accepted incoming POST requests without verifying an unpredictable, cryptographically secure anti-CSRF token.

```
<!-- VULNERABLE FORM (Pre-Remediation) -->
<form action="/delete/1" method="POST">
    <button type="submit">Delete</button>
</form>
```

4.2 Controlled Lab Demonstration (Before Fix)

An external malicious website (`tests/csrf_poc.html`) auto-submits a hidden POST request to `http://127.0.0.1:5001/delete/1` using the victim's authenticated session cookies.

Observed Result (Exploit Proof):

The server processes the forged request, deletes the victim's file, and returns `HTTP 302 Redirect`. The victim's asset is destroyed without their knowledge or authorization.

> ***[SCREENSHOT CHECKPOINT #22]:** Capture browser showing `tests/csrf_poc.html` executing a forged deletion request against Port 5001 successfully.*

4.3 Secure Remediation (After Fix)

Implemented the **Synchronizer Token Pattern** combined with strict session cookie attributes:

1. `session['csrf_token']` initialized with a cryptographically secure 256-bit random hex value (`secrets.token_hex(32)`).

2. Every state-changing form includes `<input type="hidden" name="csrf_token" value="{{ csrf_token }}">`.

3. Server-side validation via `secrets.compare_digest()` prevents timing attacks.

4. Cookie policy enforced: `SESSION_COOKIE_SAMESITE = 'Strict'` and `SESSION_COOKIE_HTTPONLY = True`.

```
# SECURE HARDENED CODE (app.py)
@app.before_request
def ensure_csrf_token():
    if "csrf_token" not in session:
        session["csrf_token"] = secrets.token_hex(32)

def validate_csrf():
    token = request.form.get("csrf_token") or request.headers.get("X-CSRF-Token")
    expected = session.get("csrf_token")
    if not token or not expected or not secrets.compare_digest(token, expected):
        return False
    return True

@app.route("/delete/<int:file_id>", methods=["POST"])
def delete_file(file_id):
    if not validate_csrf():
        return Response("403 Forbidden: CSRF token missing or invalid", status=403)
    ...
```

4.4 Post-Fix Verification

The external exploit page attempts to send a forged request to `http://127.0.0.1:5000/delete/1`.

Observed Result (Remediation Proof):

```
HTTP/1.1 403 Forbidden
Content-Type: text/plain
403 Forbidden: CSRF token missing or invalid
```

The forged request is rejected before any application state or database alteration can take place.

> ***[SCREENSHOT CHECKPOINT #23]:** Capture browser showing `HTTP 403 Forbidden: CSRF token missing or invalid` upon submitting an unauthenticated forged request.*

---

5. Vulnerability Class 5: Clickjacking / UI Redressing (CWE-1021)

5.1 Technical Vulnerability Description

In the pre-remediation baseline, HTTP responses did not specify framing restriction headers (`X-Frame-Options` or `Content-Security-Policy: frame-ancestors`). Consequently, any third-party malicious website could embed the application inside a transparent iframe, overlaying deceptive elements to trick the user into clicking administrative or file deletion buttons.

5.2 Controlled Lab Demonstration (Before Fix)

The test harness `tests/clickjacking_poc.html` embeds the pre-remediation app (`http://127.0.0.1:5001/`) inside an `<iframe>`.

Observed Result (Exploit Proof):

The browser successfully renders the full web application inside the iframe without restriction.

> ***[SCREENSHOT CHECKPOINT #24]:** Capture browser showing `tests/clickjacking_poc.html` with the vulnerable application successfully loaded inside the left iframe.*

5.3 Secure Remediation (After Fix)

Implemented global HTTP response security headers via Flask's `@app.after_request` hook, enforcing both legacy and modern standards:

```
# SECURE HARDENED CODE (app.py)
@app.after_request
def apply_security_headers(response):
    # Modern standard: CSP frame-ancestors 'none'
    response.headers["Content-Security-Policy"] = "default-src 'self'; frame-
ancestors 'none';"
    # Legacy standard: X-Frame-Options DENY
    response.headers["X-Frame-Options"] = "DENY"
    # MIME sniffing mitigation
    response.headers["X-Content-Type-Options"] = "nosniff"
    # Legacy browser XSS protection
    response.headers["X-XSS-Protection"] = "1; mode=block"
    # Referrer privacy
    response.headers["Referrer-Policy"] = "strict-origin-when-cross-origin"
    return response
```

5.4 Post-Fix Verification

The test harness attempts to embed `http://127.0.0.1:5000/` inside an `<iframe>`.

Observed Result (Remediation Proof):

The browser immediately refuses to display the framed document.

Console Output:

```
Refused to display 'http://127.0.0.1:5000/' in a frame because it set 'X-Frame-
Options' to 'DENY'.
Refused to frame 'http://127.0.0.1:5000/' because an ancestor violates the following
Content Security Policy directive: "frame-ancestors 'none'".
```

> ***[SCREENSHOT CHECKPOINT #25]:** Capture browser showing `tests/clickjacking_poc.html` with the right iframe blocked by browser security controls and developer tools console error visible.*

---

6. Summary of Screenshot Checkpoints for Final Report

| Checkpoint ID | Required Screenshot Content | File / Command |

| :--- | :--- | :--- |

| **#16** | Buffer Overflow Exploit: Stack smashing crash output | `.\parser_vulnerable.exe` |

| **#17** | Buffer Overflow Fix: Graceful bounds rejection output | `.\parser_secure.exe` |

| **#18** | Command Injection Exploit: `LAB_MARKER` executed in stdout | `processor_vulnerable.py` |

| **#19** | Command Injection Fix: Whitelist rejection & clean exit | `processor.py` |

| **#20** | IDOR Exploit: Bob accessing Alice's confidential file | Port 5001 `/download/1` |

| **#21** | IDOR Fix: HTTP 404 Access Denied on cross-user file | Port 5000 `/download/101` |

| **#22** | CSRF Exploit: Silent deletion of victim file without token | `tests/csrf_poc.html` on 5001 |

| **#23** | CSRF Fix: HTTP 403 Forbidden on missing CSRF token | Port 5000 `/delete/1` |

| **#24** | Clickjacking Exploit: Vulnerable app rendered inside iframe | `tests/clickjacking_poc.html` (Left) |

| **#25** | Clickjacking Fix: Framing blocked with X-Frame-Options DENY | `tests/clickjacking_poc.html` (Right) |

| **#26** | Automated Test Suite: All 5 vulnerability proofs passing (OK) | `python -m unittest tests/verify_all_vulnerabilities.py` |

| **#27** | Bandit SAST Before: 2 High severity findings on vulnerable code | `bandit -r app_vulnerable.py processor_vulnerable.py` |

| **#28** | Bandit SAST After: 0 High / 0 Medium findings on secure code | `bandit -r app.py processor.py -lll` |

| **#29** | GitHub Actions Pipeline: Green passing CI/CD run with all gates | GitHub Actions Web UI |

| **#30** | Jira Sprint Board: All Epics, Stories, and Tasks in Done state | Jira Cloud `SASS` Board |

Report Compiled & Verified by: Talal Nadeem Awan (FA23-BCT-051) & Shahroze Afaq (FA23-BCT-039)